DS9 Integrated Source Extractor

Technical Reference Manual

SEP-Based Source Detection, Photometry,
and Catalog Management
Integrated into SAOImageDS9

Juhan Kim

March 2026

Based on SAOImageDS9 (Smithsonian Astrophysical Observatory)
and SEP — Source Extraction and Photometry (Barbary et al.)

Contents

User Guide

1 Quick Start Guide

This chapter provides a step-by-step guide to using the integrated Source Extractor in DS9. For detailed explanations of the extraction algorithm, output parameters, and internal architecture, refer to the subsequent chapters.

1.1 Overview

The DS9 Integrated Source Extractor adds SExtractor-like source detection and photometry capabilities directly within the SAOImageDS9 image viewer. When a FITS image is loaded, sources can be extracted with a single click, and the resulting catalog is displayed in a resizable panel alongside the image.

Key features:

- **One-click extraction** — Press the “Extract” button to detect and measure all sources in the current FITS image
- **Split-panel interface** — The main window is divided into an image viewer (left) and a catalog table (right), separated by a draggable divider
- **19 output parameters** — Photometry, astrometry, morphology, surface brightness, and star/galaxy classification
- **Real-time filtering** — Filter the catalog interactively by any column value
- **Automatic WCS** — Celestial coordinates (RA, Dec) are computed automatically when WCS information is available in the FITS header
- **Auto-extraction** — Sources are extracted automatically when a new FITS file is loaded

1.2 Prerequisites

- **DS9** — SAOImageDS9 built with the integrated Source Extractor patch
- **Python 3** with the following packages:
  - `sep` — Source Extraction and Photometry library (Python binding of SExtractor)
  - `astropy` — FITS I/O and WCS coordinate transformations
  - `numpy` — Numerical array operations
- `ds9_sextract` **binary** — The compiled Python script, located in the same directory as the `ds9` executable

Install Python Dependencies

```
pip install sep astropy numpy
```

1.3 Building DS9 with the Source Extractor

Build Commands

```
cd SAOImageDS9
make -C ds9/unix -j 128
```

The build process compiles the C/Tcl core, packages the Tcl library files (including the modified `layout.tcl` and `ds9.tcl`) into `ds9.zip`, and produces the final `bin/ds9` executable. The `ds9_sextract` binary must be placed in the same `bin/` directory.

Important

After modifying any `.tcl` file in `ds9/library/`, you must copy it to `ds9/unix/mntpt/library/`, update the zip, and rebuild:

```
cp ds9/library/layout.tcl ds9/unix/mntpt/library/
cp ds9/library/ds9.tcl     ds9/unix/mntpt/library/
cd ds9/unix
zip -ru ds9.zip mntpt/library/layout.tcl mntpt/library/ds9.
    tcl
cat ds9base ds9.zip > ds9
zip -A ds9
chmod 755 ds9
cp ds9 ../../bin/
```

1.4 Using the Source Extractor

1.4.1 Step 1: Launch DS9

Launch

```
./bin/ds9 image.fits
```

The window opens with two panels separated by a vertical divider:

- **Left panel** — The standard DS9 interface (menu bar, info panel, buttons, image canvas, colorbar)
- **Right panel** — The Source Extractor catalog panel (title bar, filter bar, data table, status bar)

The divider can be dragged left or right to resize the panels.

1.4.2 Step 2: Extract Sources

Click the **"Extract"** button in the catalog panel title bar. The status bar will show progress:

1. `Extracting sources from image.fits ...` — Extraction is running
2. `image.fits: 142 sources extracted` — Extraction complete

The catalog table is automatically populated with all detected sources, sorted by brightness (brightest first).

Auto-Extraction

When a FITS file is loaded via `File` → `Open`, sources are extracted automatically after a 500 ms delay. This behavior is controlled by the `CatalogPanelAutoExtract` hook in `fits.tcl`.

1.4.3 Step 3: Browse the Catalog

The catalog table displays 19 columns (see Chapter 4 for detailed descriptions):

Category	Columns
Identification	NUMBER
Astrometry	X_IMAGE, Y_IMAGE, ALPHA_J2000, DELTA_J2000
Photometry	MAG_AUTO, MAG_ISOCOR, MAG_APER, FLUX_AUTO
Morphology	FLUX_RADIUS, A_IMAGE, B_IMAGE, THETA_IMAGE, ELLIPTICITY KRON_RADIUS
Surface Brightness	MU_MAX, MU_THRESHOLD
Classification	CLASS_STAR, FLAGS

- Scroll horizontally and vertically using the scrollbars
- Column widths can be resized by dragging column borders in the header row
- Rows can be selected by clicking (extended selection with Shift/Ctrl)

1.4.4 Step 4: Filter Results

Type a search pattern in the **Filter** entry and click **"Apply"** (or press Enter). The filter performs case-insensitive substring matching across all columns. For example:

- Type `0.9` to show only rows containing "0.9" in any field
- Clear the filter field and press Apply to show all sources again

1.4.5 Step 5: Clear and Re-extract

Click **"Clear"** to reset the table and status. Load a new FITS image and click **"Extract"** again, or rely on auto-extraction.

1.5 Interface Layout

The window hierarchy below the pulldown menu bar:

Widget Hierarchy

```
. (top-level window)
|-- .mb (pulldown menu bar: File, Edit, View, ...)
|-- .pw (ttk::panedwindow, horizontal, draggable divider)
    |-- .ds9 (left pane: main DS9 interface)
    |    |-- header (info panel, magnifier, panner)
    |    |-- buttons (toolbar)
    |    |-- image frame
    |        |-- canvas (FITS image display)
    |        |-- colorbar, graphs
    |-- .catf (right pane: catalog panel)
        |-- titlebar ("Source Extractor", Extract, Clear)
```

```
            |-- searchbar (Filter entry + Apply)
            |-- table (19-column scrollable data table)
            |-- statusbar (status messages)
```

The `ttk::panedwindow` widget provides a sash (divider handle) that the user can drag to redistribute space between the image viewer and the catalog panel. The left pane has weight 3 and the right pane has weight 2, giving an initial 60:40 split.

1.6 Troubleshooting

Symptom	**Solution**
`ds9_sextract not found`	Ensure the `ds9_sextract` binary is in the same directory as the `ds9` executable. Check that it has execute permission (`chmod +x`).
`No FITS image loaded`	Open a FITS file before clicking Extract. The extractor requires a frame with a loaded FITS image.
`Extraction error: ...`	Check that Python dependencies (`sep`, `astropy`, `numpy`) are installed. Check `LD_LIBRARY_PATH` if using conda.
`No sources detected`	The image may have very low signal-to-noise. Try adjusting the detection threshold (default: 1.5σ) via command-line arguments to `ds9_sextract`.
Catalog panel not visible	Drag the divider from the right edge of the window toward the center. The panel may have been collapsed.
WCS coordinates show 0.0	The FITS header does not contain valid WCS keywords. Pixel coordinates (X_IMAGE, Y_IMAGE) are still accurate.

2 Architecture

This chapter describes the internal architecture of the integrated Source Extractor, covering the modified DS9 source files, the extraction pipeline, and the data flow between components.

2.1 Modified Source Files

Three files implement the integration:

File	Role
`ds9/library/ds9.tcl`	Creates the top-level `ttk::panedwindow` that splits the main window into two panes. Initializes `ds9(toppw)`, `ds9(main)`, and `ds9(catalog_frame)`. Calls `CreateCatalogPanel`.
`ds9/library/layout.tcl`	Defines `CreateCanvas` (image canvas, restored to original structure), `CreateCatalogPanel` (catalog UI), `CatalogPanelExtract` (runs `ds9_sextract`), `CatalogPanelLoadTSV` (parses output into table), `CatalogPanelFilter` (interactive filtering), and `CatalogPanelAutoExtract` (auto-extraction hook).
`ds9/library/fits.tcl`	Calls `CatalogPanelAutoExtract` after a FITS file is loaded, triggering automatic source extraction.

2.2 Top-Level Window Structure

The key architectural change is the introduction of a `ttk::panedwindow` at the highest level of the widget hierarchy, directly below the native pulldown menu bar.

ds9.tcl — Top-level panedwindow creation

```
# create top-level panedwindow
set ds9(toppw) [ttk::panedwindow ${ds9(top)}pw -orient
    horizontal]
pack $ds9(toppw) -fill both -expand true

# left pane: main DS9 content
set ds9(main) [ttk::frame ${ds9(top)}ds9]
```

```
$ds9(toppw) add $ds9(main) -weight 3

# right pane: catalog panel
set ds9(catalog_frame) [ttk::frame ${ds9(top)}catf]
$ds9(toppw) add $ds9(catalog_frame) -weight 2
```

This design ensures that:

1. The pulldown menu bar spans the full window width (it is set via `$ds9(top) configure -menu $ds9(mb)`, independent of pack geometry)
2. Everything below the menu bar is split into two resizable panes
3. All existing DS9 layout modes (horizontal, vertical, basic, advanced) continue to work within the left pane without modification
4. The catalog panel occupies its own independent pane on the right

2.3 Data Flow

The extraction pipeline follows this sequence:

1. **Trigger** — User clicks “Extract” (or auto-extraction fires after FITS load)
2. **File discovery** — `CatalogPanelExtract` retrieves the current FITS filename from the active frame via `$current(frame) get fits file name full`
3. **External process** — The `ds9_sextract` binary is executed as a child process via `exec sh -c`. The `LD_LIBRARY_PATH` is configured to include conda library paths.
4. **Output capture** — `ds9_sextract` writes a tab-separated table to `stdout`. The Tcl `exec` command captures this output as a string.
5. **Table population** — `CatalogPanelLoadTSV` parses the TSV string, splits it into rows and columns, and populates the `Tktable` widget via the `catpaneltbldb` array variable.
6. **Status update** — The status bar displays the filename and number of detected sources.

> The extraction runs as a separate process, not within the DS9 Tcl interpreter. This prevents numerical library conflicts and allows the Python-based extractor to use its own dependencies (NumPy, SEP, Astropy) independently.

3 Extraction Algorithm

The `ds9_sextract` tool implements a SExtractor-compatible extraction pipeline using the SEP (Source Extraction and Photometry) library. This chapter describes each step of the algorithm in detail.

3.1 Background Estimation

The first step is to estimate and subtract the sky background. SEP uses the same algorithm as SExtractor:

1. The image is divided into a grid of rectangular meshes (default size: 64×64 pixels, controlled by `-back-size`)
2. In each mesh cell, the local background level is estimated using iterative $\kappa\sigma$-clipping
3. The resulting coarse background map is smoothed with a median filter (default kernel: 3×3 cells, controlled by `-back-filtersize`)
4. The smoothed map is interpolated to full image resolution using bicubic spline interpolation

Background estimation

```
bkg = sep.Background(data, bw=back_size, bh=back_size,
                     fw=back_filtersize, fh=back_filtersize)
data_sub = data - bkg.back()
```

Two outputs are produced:

- `bkg.back()` — the background level map $B(x,y)$
- `bkg.rms()` — the background RMS (noise) map $\sigma_B(x,y)$, used as the detection threshold and photometric error

3.2 Source Detection

Sources are detected as groups of connected pixels above a threshold:

$$I(x,y) - B(x,y) > k \cdot \sigma_B(x,y) \tag{3.1}$$

where k is the detection threshold in units of the local background RMS (default: $k = 1.5$, set by `-detect-thresh`). Connected pixel groups with fewer than `-detect-minarea` pixels (default: 5) are rejected.

3.2.1 Observation Boundary Masking

Many astronomical images—particularly JWST and HST mosaics—contain regions outside the actual observation footprint that are filled with zero-valued pixels. Without masking, SEP may detect spurious sources at the boundary between the observed and unobserved regions, where the sharp transition from zero to real flux mimics a source profile.

ds9_sextract automatically constructs a **pixel mask** from the original FITS data before any processing:

$$M(x,y) = \begin{cases} 1 & \text{if } I_{\text{raw}}(x,y) = 0, \text{ NaN, or } \pm\infty \\ 0 & \text{otherwise} \end{cases} \tag{3.2}$$

This mask is passed to SEP via the sep_image.mask field, which excludes masked pixels from both **background estimation** and **source detection**. This prevents the zero-filled regions from biasing the local background model and eliminates most false detections at the observation boundary.

Boundary Overlap Filtering

After source detection and measurement, a second filter removes any source whose **isophotal footprint** overlaps with the mask. For each source, the isophotal ellipse is defined by:

- Semi-major axis: ISO_RADIUS
- Axis ratio: b/a from B_IMAGE / A_IMAGE
- Position angle: THETA_IMAGE

Every pixel inside this ellipse is checked against the mask. If any masked pixel falls within the isophotal ellipse, the source is excluded from the output catalog. This ensures that partially observed sources at the observation boundary—which would have unreliable photometry and morphology—are not included in the final catalog.

The number of filtered sources is reported to stderr:

```
Masked 5765432 / 27562500 pixels (outside observation)
Filtered: 342 / 7204 sources outside observation boundary
```

3.2.2 Deblending

Overlapping sources are separated using multi-threshold deblending:

1. The detected object is re-thresholded at -deblend-nthresh (default: 32) exponentially spaced sub-thresholds between the detection threshold and the peak value
2. At each sub-threshold, connected components are identified
3. A component is split into a separate object if it contains more than -deblend-mincont (default: 0.005) of the total flux of the parent object

Source detection with deblending

```
objects, segmap = sep.extract(
    data_sub, detect_thresh, err=bkg_rms,
    minarea=detect_minarea,
    deblend_nthresh=deblend_nthresh,
    deblend_cont=deblend_mincont,
    segmentation_map=True
)
```

The segmentation map assigns each pixel to the object it belongs to (or zero for background), enabling morphological measurements.

3.3 Photometry

Three photometric methods are applied to each detected source.

3.3.1 AUTO Photometry (Kron Elliptical Aperture)

The AUTO magnitude follows the Kron (1980) method:

1. The first-moment radius (Kron radius r_K) is computed within an ellipse of semi-major axis $6a$:

   $$r_K = \frac{\sum_i r_i I_i}{\sum_i I_i} \tag{3.3}$$

   where r_i is the elliptical distance of pixel i from the object center, and I_i is the background-subtracted intensity.
2. A minimum Kron radius of 3.5 pixels is enforced to prevent underestimation for compact sources.
3. The flux is summed within an elliptical aperture of semi-major axis $2.5\,r_K$, following the SExtractor convention.

Kron Factor

The factor of 2.5 applied to the Kron radius ensures that $\gtrsim$96% of the total flux is captured for a typical galaxy profile (Sérsic $n \approx 1$–4), while $\gtrsim$99% is captured for a Gaussian PSF.

3.3.2 Isophotal Corrected Photometry

The isophotal flux is the sum of all pixels belonging to the detection footprint. A correction factor accounts for the flux missed below the detection isophote, using the SExtractor formula:

$$F_{\text{isocor}} = \frac{F_{\text{iso}}}{1 - 0.196\,\alpha - 0.751\,\alpha^2}, \qquad \alpha = \frac{N_{\text{pix}} \cdot t_{\text{local}}}{F_{\text{iso}}} \tag{3.4}$$

where N_{pix} is the number of detection pixels, t_{local} is the local threshold, and F_{iso} is the raw isophotal flux.

3.3.3 Aperture Photometry

A fixed circular aperture of diameter `-phot-aperture` (default: 5.0 pixels) is centered on each source. The flux is summed within this circle, with partial pixel weighting at the boundary.

3.3.4 Magnitude Conversion

All fluxes are converted to magnitudes using:

$$m = -2.5\log_{10}(F) + m_0 \tag{3.5}$$

where m_0 is the magnitude zero-point (default: 25.0, set by `-mag-zeropoint`). Sources with non-positive flux are assigned $m = 99.0$.

3.4 Morphological Parameters

3.4.1 Shape Parameters

The semi-major axis (a), semi-minor axis (b), and position angle (θ) are derived from the second-order moments of the intensity distribution within the detection isophote:

$$\overline{x^2} = \frac{\sum_i I_i x_i^2}{\sum_i I_i} - \bar{x}^2 \tag{3.6}$$

$$\overline{y^2} = \frac{\sum_i I_i y_i^2}{\sum_i I_i} - \bar{y}^2 \tag{3.7}$$

$$\overline{xy} = \frac{\sum_i I_i x_i y_i}{\sum_i I_i} - \bar{x}\bar{y} \tag{3.8}$$

The ellipse parameters are:

$$a^2 = \frac{\overline{x^2} + \overline{y^2}}{2} + \sqrt{\left(\frac{\overline{x^2} - \overline{y^2}}{2}\right)^2 + \overline{xy}^2} \tag{3.9}$$

$$b^2 = \frac{\overline{x^2} + \overline{y^2}}{2} - \sqrt{\left(\frac{\overline{x^2} - \overline{y^2}}{2}\right)^2 + \overline{xy}^2} \tag{3.10}$$

$$\theta = \frac{1}{2} \arctan\left(\frac{2\overline{xy}}{\overline{x^2} - \overline{y^2}}\right) \tag{3.11}$$

The ellipticity is defined as:

$$e = 1 - \frac{b}{a} \tag{3.12}$$

3.4.2 Half-Light Radius

The half-light radius (`FLUX_RADIUS`) is the radius of a circular aperture centered on the source that encloses 50% of the AUTO flux. It is computed by integrating the flux profile outward from the center until the cumulative flux reaches $0.5 \times F_{\mathrm{AUTO}}$.

3.5 Surface Brightness

Two surface brightness measurements are provided:

- **MU_MAX** — Surface brightness of the brightest pixel:

$$\mu_{\mathrm{max}} = -2.5 \log_{10}\left(\frac{I_{\mathrm{peak}}}{\Omega_{\mathrm{pix}}}\right) + m_0 \tag{3.13}$$

  where I_{peak} is the peak pixel value above background and Ω_{pix} is the pixel area in arcsec^2.
- **MU_THRESHOLD** — Surface brightness at the detection threshold:

$$\mu_{\mathrm{thresh}} = -2.5 \log_{10}\left(\frac{t_{\mathrm{local}}}{\Omega_{\mathrm{pix}}}\right) + m_0 \tag{3.14}$$

3.6 Star/Galaxy Classification

An approximate star/galaxy classifier is implemented using the ratio of the measured FWHM to the seeing FWHM:

$$\mathrm{FWHM}_{\mathrm{image}} = 2\sqrt{\ln 2 \cdot (a^2 + b^2)} \tag{3.15}$$

$$\mathrm{CLASS\_STAR} = \frac{1}{1+\exp(3\cdot(r_{\mathrm{FWHM}} - 1.5))}, \qquad r_{\mathrm{FWHM}} = \frac{\mathrm{FWHM}_{\mathrm{image}}}{\mathrm{FWHM}_{\mathrm{seeing}}/s} \tag{3.16}$$

where s is the pixel scale in arcsec/pixel and $\mathrm{FWHM}_{\mathrm{seeing}}$ is set by `-seeing-fwhm` (default: 3.0 arcsec).

Objects with CLASS_STAR ≈ 1 are point-like (stars), while CLASS_STAR ≈ 0 indicates extended sources (galaxies).

> This is a simple heuristic classifier, not the neural-network-based classifier used by SExtractor. For accurate star/galaxy separation, use dedicated tools such as `PSFEx` + SExtractor or machine learning classifiers.

3.7 Flags

The `FLAGS` column is a bitmask combining detection and photometry flags:

Bit	Value	Meaning
0	1	Object has neighbors (bright and close enough to bias photometry)
1	2	Object was originally blended with another
2	4	At least one pixel is saturated (peak + background $\geq$ `SATURATE`)
3	8	Object is truncated (too close to image boundary)
4	16	Aperture data incomplete or corrupted
5	32	Isophotal data incomplete or corrupted

Multiple flags can be set simultaneously. For example, `FLAGS = 3` means the object has neighbors (1) and was deblended (2).

3.8 WCS Coordinate Transformation

If the FITS header contains valid World Coordinate System (WCS) keywords (`CRPIX`, `CRVAL`, `CD` or `CDELT+CROTA`), pixel coordinates are converted to celestial coordinates (J2000 equatorial):

WCS transformation

```
wcs = WCS(header)
coords = wcs.pixel_to_world(objects['x'], objects['y'])
alpha_j2000 = coords.ra.deg     # Right Ascension in degrees
delta_j2000 = coords.dec.deg    # Declination in degrees
```

If WCS is unavailable, `ALPHA_J2000` and `DELTA_J2000` are set to 0.0.

4 Output Parameters

This chapter provides a detailed description of each column in the output catalog.

4.1 Complete Parameter Reference

Parameter	Unit	Description
NUMBER	—	Running object number, assigned after sorting by brightness (brightest = 1).
X_IMAGE	pixel	Object centroid x-coordinate in FITS convention (1-indexed). Computed as the intensity-weighted first moment.
Y_IMAGE	pixel	Object centroid y-coordinate in FITS convention (1-indexed).
ALPHA_J2000	deg	Right Ascension (J2000 equatorial) computed from pixel coordinates via WCS. Zero if no WCS is available.
DELTA_J2000	deg	Declination (J2000 equatorial) computed from pixel coordinates via WCS. Zero if no WCS is available.
MAG_AUTO	mag	Kron-like elliptical aperture magnitude. Uses an elliptical aperture with semi-major axis $= 2.5 \times r_{\mathrm{Kron}}$. Most reliable total magnitude for galaxies.
MAG_ISOCOR	mag	Isophotal magnitude corrected for flux lost below the detection threshold, using the SExtractor analytical correction formula.
MAG_APER	mag	Fixed circular aperture magnitude. Aperture diameter set by `-phot-aperture` (default: 5.0 pixels). Useful for comparing point sources.
FLUX_AUTO	counts	Total flux within the Kron elliptical aperture (in ADU or counts).

Parameter	Unit	Description
FLUXERR_AUTO	counts	1σ flux uncertainty for FLUX_AUTO, from Poisson + background noise.
FLUXERR_APER	counts	1σ flux uncertainty for MAG_APER.
MAGERR_AUTO	mag	Magnitude error for MAG_AUTO: $\sigma_m = 1.0857 \times \text{FLUXERR\_AUTO}/\text{FLUX\_AUTO}$.
MAGERR_APER	mag	Magnitude error for MAG_APER.
FLUX_APER_2	counts	Flux in the 2nd circular aperture (diameter set by -phot-aperture-2, default 4.0 px).
FLUX_APER_3	counts	Flux in the 3rd circular aperture (diameter set by -phot-aperture-3, default 6.0 px).
FLUX_APER_5	counts	Flux in the 4th circular aperture (diameter set by -phot-aperture-5, default 10.0 px).
FLUXERR_APER_2/3/5	counts	1σ flux errors for the corresponding multi-aperture measurements.
MAG_APER_2/3/5	mag	Magnitudes for the corresponding multi-aperture measurements.
FLUX_RADIUS	pixel	Half-light radius: radius of a circular aperture enclosing 50% of FLUX_AUTO. Indicator of object size; correlates with effective radius R_e for galaxies.
A_IMAGE	pixel	Semi-major axis of the object ellipse, derived from second-order intensity moments.
B_IMAGE	pixel	Semi-minor axis of the object ellipse.
THETA_IMAGE	deg	Position angle of the major axis, measured counter-clockwise from the x-axis (FITS convention). Range: -90 to $+90$.
ELLIPTICITY	—	Ellipticity: $e = 1 - b/a$. Values range from 0 (circular) to 1 (infinitely elongated).
KRON_RADIUS	pixel	First-moment radius (Kron radius), used to scale the AUTO aperture. Minimum enforced value: 3.5 pixels.
FWHM_IMAGE	pixel	Full Width at Half Maximum, computed as $2\sqrt{\ln 2 \cdot (a^2 + b^2)}$ from the intensity-weighted second moments. Used internally for star/galaxy classification.
ISO_RADIUS	pixel	Equivalent semi-major axis of an ellipse with the same area as the isophotal detection footprint.
NPIX_ISO	pixel^2	Number of pixels in the isophotal detection footprint above the detection threshold.

Parameter	Unit	Description
MU_MAX	mag/arcsec^2	Peak surface brightness: magnitude of the brightest pixel divided by pixel area. Lower values = brighter peaks.
MU_THRESHOLD	mag/arcsec^2	Detection threshold surface brightness at the object position. Objects with $\mu_{\text{max}} < \mu_{\text{thresh}}$ are detected.
CLASS_STAR	—	Star/galaxy classifier output. Range 0.0 (extended/galaxy) to 1.0 (point-like/star). Based on FWHM ratio to seeing.
FLAGS	—	Extraction flags (bitmask). See Section 3 for flag definitions. Zero means clean photometry.

5 Command-Line Reference

The ds9_sextract tool can also be used as a standalone command-line program, independent of the DS9 GUI.

5.1 Synopsis

```
ds9_sextract [OPTIONS] <FITS_FILE>
```

The output is a tab-separated table printed to stdout. The first line is the header row containing column names; subsequent lines contain data rows sorted by MAG_AUTO (brightest first).

5.2 Options

Option	Default	Description
-detect-thresh	1.5	Detection threshold in units of the local background RMS (σ). Lower values detect fainter sources but increase false positives.
-detect-minarea	5	Minimum number of connected pixels above the threshold required to register a detection. Rejects cosmic ray hits and noise spikes.
-deblend-nthresh	32	Number of exponentially spaced sub-thresholds used for deblending overlapping sources. Higher values provide finer deblending.
-deblend-mincont	0.005	Minimum contrast ratio for deblending. A sub-component must contain at least this fraction of the parent flux to be split off. Lower values split more aggressively.
-phot-aperture	5.0	Diameter of the fixed circular aperture (in pixels) used for MAG_APER measurements.

Option	Default	Description
`-mag-zeropoint`	25.0	Photometric zero-point m_0 for magnitude conversion: $m = -2.5\log_{10}(F) + m_0$. Set this to the calibrated zero-point of your image.
`-gain`	0	Detector gain in e^-/ADU. Used for Poisson noise estimation in photometric errors. If 0, the tool attempts to read `GAIN`, `EGAIN`, or `CCDGAIN` from the FITS header.
`-pixel-scale`	1.0	Pixel scale in arcsec/pixel. Used for surface brightness calculations. If 1.0 and WCS is available, the scale is automatically derived from WCS.
`-seeing-fwhm`	3.0	Seeing FWHM in arcsec. Used for star/galaxy classification (`CLASS_STAR`).
`-back-size`	64	Background mesh size in pixels. Each mesh cell has dimensions `back-size` × `back-size`. Smaller values track finer background variations but may subtract source flux.
`-back-filtersize`	3	Median filter kernel size (in mesh cells) applied to the background map before interpolation. Smooths out mesh-to-mesh noise.
`-phot-aperture-2`	4.0	Diameter (pixels) of the 2nd circular aperture for multi-aperture photometry (`FLUX_APER_2`, `MAG_APER_2`).
`-phot-aperture-3`	6.0	Diameter of the 3rd circular aperture.
`-phot-aperture-5`	10.0	Diameter of the 4th circular aperture.
`-conv-filter`	default	Convolution filter for source detection. Options: `default` (3×3 Gaussian), `gauss5x5` (5×5 Gaussian), `mexhat` (5×5 Mexican hat), `tophat` (5×5 top-hat).

5.3 Usage Examples

Basic extraction

```
ds9_sextract image.fits > catalog.tsv
```

Deep extraction with calibrated zero-point

```
ds9_sextract --detect-thresh 1.0 --detect-minarea 3 \
             --mag-zeropoint 28.09 --gain 2.5 \
```

```
              --seeing-fwhm 0.8 --pixel-scale 0.03 \
              hst_acs_f814w.fits > deep_catalog.tsv
```

Conservative extraction (few false positives)

```
ds9_sextract --detect-thresh 3.0 --detect-minarea 10 \
             --deblend-mincont 0.01 \
             crowded_field.fits > clean_catalog.tsv
```

Fine background estimation for extended objects

```
ds9_sextract --back-size 128 --back-filtersize 5 \
             galaxy_cluster.fits > cluster_catalog.tsv
```

II

Developer Reference

6 Tcl/Tk Implementation Details

This chapter documents the Tcl procedures that implement the catalog panel and its integration with the DS9 GUI.

6.1 Catalog Panel Procedures

6.1.1 CreateCatalogPanel

Creates all widgets for the catalog panel within `ds9(catalog_frame)`:

Widget	**Description**
`titlebar`	Horizontal frame containing the “Source Extractor” label, “Extract” button, and “Clear” button.
`searchbar`	Filter entry with “Apply” button. The entry is bound to `<Return>` for keyboard activation.
`tblf` (table frame)	Contains the `Tktable` widget (`catpanel(tbl)`) with horizontal and vertical scrollbars. The table is configured with 19 columns, row selection mode, and resizable column borders.
`statusbar`	Displays status messages via the `catpanel(status)` variable.

6.1.2 CatalogPanelExtract

Orchestrates the extraction process:

1. Locates the `ds9_sextract` binary in the same directory as the DS9 executable
2. Retrieves the current FITS filename from the active frame
3. Constructs `LD_LIBRARY_PATH` to include conda libraries (both `$CONDA_PREFIX/lib` and `$HOME/miniconda3/lib`)
4. Executes `ds9_sextract` via `exec sh -c` and captures stdout
5. Passes the TSV output to `CatalogPanelLoadTSV`

6.1.3 CatalogPanelLoadTSV

Parses tab-separated data into the Tktable widget:

1. Splits the input string by newlines into rows
2. Parses the first row as column headers
3. Configures the table dimensions (`-cols`, `-rows`)
4. Fills the `catpaneltbldb` array variable: `catpaneltbldb(`*row*`,`*col*`)`
5. Stores the raw data in `catpanel(alldata)` for filtering

6.1.4 CatalogPanelFilter

Performs case-insensitive substring filtering:

1. Reads the filter pattern from `catpanel(search_var)`
2. Iterates over stored data rows (`catpanel(alldata)`)
3. Includes a row if the pattern appears anywhere in the tab-separated line (`string match -nocase`)
4. Repopulates the table with matching rows only
5. Updates the status bar with the count of matching rows

6.1.5 CatalogPanelAutoExtract

A hook procedure called from `fits.tcl` after a FITS file is loaded:

```
proc CatalogPanelAutoExtract {} {
    global catpanel
    if {[info exists catpanel(tbl)]} {
        after 500 CatalogPanelExtract
    }
}
```

The 500 ms delay (`after 500`) ensures the image is fully loaded and rendered before extraction begins.

6.2 Global Variables

Variable	Purpose
`ds9(toppw)`	The top-level `ttk::panedwindow` widget path
`ds9(catalog_frame)`	Frame widget path for the right (catalog) pane
`catpanel(tbl)`	The Tktable widget path
`catpanel(tbldb)`	Name of the global array backing the table data
`catpanel(status)`	Status bar text variable
`catpanel(search_var)`	Filter entry text variable
`catpanel(alldata)`	Raw TSV string from last extraction (for filtering)
`catpanel(filename)`	Last extracted FITS filename
`catpanel(delim)`	Column delimiter (always "`\t`")

7 Python Extractor Details

7.1 Module Structure

The ds9_sextract script (ds9_sextract.py) is a single-file Python module with one main function:

```
def extract_sources(fits_file,
                    detect_thresh=1.5,
                    detect_minarea=5,
                    deblend_nthresh=32,
                    deblend_mincont=0.005,
                    phot_aperture=5.0,
                    mag_zeropoint=25.0,
                    gain=0.0,
                    pixel_scale=1.0,
                    seeing_fwhm=3.0,
                    back_size=64,
                    back_filtersize=3):
```

7.2 FITS File Handling

The FITS reader searches for a 2D image HDU in the following order:

1. Iterate through all HDUs; use the first one with 2D data (`hdu.data.ndim == 2`)
2. If no 2D HDU is found, try the primary HDU
3. If the primary has > 2 dimensions, extract the first 2D slice (`data[0]`)
4. The data is converted to `float64` C-contiguous format (required by SEP)

7.3 Automatic Header Parsing

Several parameters are automatically read from the FITS header when not specified by the user:

Parameter	Header Keywords	Fallback
Gain	GAIN, EGAIN, CCDGAIN	0 (Poisson noise disabled)
Saturation level	SATURATE, SATUR, SATULEVEL, DATAMAX	65535
Pixel scale	WCS proj_plane_pixel_scales()	1.0 arcsec/pixel

7.4 Dependencies

Package	Version	Purpose
`sep`	≥ 1.0	Core source extraction: background estimation, detection, deblending, photometry, and morphology. Python binding of the C library derived from SExtractor.
`astropy`	≥ 4.0	FITS file I/O (`astropy.io.fits`) and WCS coordinate transformations (`astropy.wcs.WCS`).
`numpy`	≥ 1.20	Array operations, mathematical functions, and data type management.

8 Image–Catalog Interaction

This chapter describes the interactive features that link the image display with the catalog panel, enabling bidirectional navigation, source merging, and advanced filtering.

8.1 Overview of Interaction Features

The following features connect the image markers to the catalog table:

Feature	Description
Mark All	Places yellow ellipse markers on all detected sources in the image
Marker Click	Clicking a yellow marker in the image selects and scrolls to the corresponding row in the catalog table, and shows cyan cross + green ellipse markers at the source position
Visible Filter	Toggles display to show only sources within the current viewport
Ctrl+Click Merge	Ctrl+Click on markers to select merge candidates (red thick ellipses); Ctrl+M merges them into a single source
Trim	Column-based min/max filtering with a dialog for each catalog column
Settings	Adjustable extraction parameters (threshold, deblending, photometry, etc.)

8.1.1 Title Bar Layout

```
[SExtractor] [Display] [AI Merge] [Galaxy Model]
[Star(PSF)] [Deconvolution] [Separate] [Analysis]
```

The eight menu buttons provide access to all features:

- **SExtractor** — Extract, Dual-Image Extract, Settings, Trim, Save/Load/Export Catalog, Export Regions, Export FITS Table, Clear

- **Display** — Mark All, Clear Markers, Show Visible Only, Add Objects (Click + A), Delete Selected (Click + D)
- **AI Merge** — Run AI Merge (MLP-based source deblending)
- **Galaxy Model** — AI Morphology Classification, Elliptical/Spiral Model Fitting
- **Star(PSF)** — Star Finding (combined/class_star/fwhm), Build PSF (median/mean/epsf/gaussian/moffat), View/Save/Load PSF, Settings
- **Deconvolution** — Deconvolve (rl/rl_accelerated/rl_tv/wiener/tikhonov/clean/mem), Quick Deconvolve, Settings
- **Separate** — Separate selected source into sub-components, Settings, Save/Load
- **Analysis** — Non-Parametric Morphology, Sérsic Fitting, PSF Photometry, Multi-Band Photometry, Crowded Field Photometry, Cross-Match (VizieR), Segmentation Map, Completeness Simulation

8.2 Mark All Sources

The **Mark All** button creates yellow ellipse markers on all detected sources using the `ISO_RADIUS`, `B_IMAGE/A_IMAGE` ratio, and `THETA_IMAGE` parameters:

Marker region string

```
ellipse($x $y ${semi_a}i ${semi_b}i $theta)
  # color=yellow width=1 tag={sextract_all} tag={
     sextract_src.$num}
    select=0 edit=0 move=0 rotate=0 delete=1 highlite=1
    callback=highlite CatalogPanelMarkerCB {$num}
    callback=unhighlite CatalogPanelMarkerUnCB {$num}
```

Each marker has:

- `sextract_all` tag for bulk operations (deletion, etc.)
- `sextract_src.`*`NUMBER`* tag for individual identification
- `highlite=1` with callbacks for mouse interaction in `catalog` mode

8.3 Marker Click — Image to Catalog Navigation

Clicking a yellow ellipse marker in the image navigates the catalog to the corresponding source.

8.3.1 Implementation

DS9's built-in `highlite` callbacks only fire in `catalog` mode. Since the default viewing mode is `none`, explicit click handlers were added in `frame.tcl`:

frame.tcl — Button1Frame (none mode)

```
none {
    if {$which == $current(frame)} {
        CatalogPanelMarkerClick $which $x $y
    } else {
        ...
    }
}
```

The click detection uses DS9's marker API:

CatalogPanelMarkerClick

```
proc CatalogPanelMarkerClick {which x y} {
    set id [$which get marker catalog id $x $y]
    if {$id == 0} return
    set tags [$which get marker catalog $id tag]
    foreach tag $tags {
        if {[string match "sextract_src.*" $tag]} {
            set src_num [string range $tag 13 end]
            break
        }
    }
    CatalogPanelMarkerCB $src_num $id
}
```

CatalogPanelMarkerCB then:

1. Finds the NUMBER column in the table header
2. Searches all rows for the matching source number
3. Selects the row and scrolls the table to it
4. Calls CatalogPanelGotoSource to pan the image and show selection markers (cyan cross + green ellipse)

8.4 Visible Filter

The **Visible** button toggles between showing all sources and showing only sources within the current viewport.

8.4.1 Viewport Calculation

Viewport bounds computation

```
set center [$frame get cursor image]    ;# {cx cy}
set zoom   [$frame get zoom]            ;# {zx zy}
set cw [winfo width $ds9(canvas)]
set ch [winfo height $ds9(canvas)]
set x_min [expr {$cx - $cw / 2.0 / $zx}]
set x_max [expr {$cx + $cw / 2.0 / $zx}]
set y_min [expr {$cy - $ch / 2.0 / $zy}]
set y_max [expr {$cy + $ch / 2.0 / $zy}]
```

Sources with X_IMAGE and Y_IMAGE within these bounds are displayed. The status bar shows "Visible: N of M sources in current view."

8.5 Source Merging (Ctrl+Click + Ctrl+M)

Multiple sources can be merged into a single source through a two-step process.

8.5.1 Step 1: Select Merge Candidates

Ctrl+Click on a yellow ellipse marker toggles it as a merge candidate:

- Selected: A red thick ellipse (width=3) is overlaid, tagged sextract_merge and sextract_merge.*NUMBER*
- Deselected: Ctrl+Click again removes the red marker and the source from the merge list

- Status bar: “Merge: N sources selected (Ctrl+M to merge, Esc to cancel)”

8.5.2 Step 2: Execute Merge (Ctrl+M)

When at least 2 sources are selected, **Ctrl+M** merges them:

Property	Merge Algorithm
`X_IMAGE`, `Y_IMAGE`	Flux-weighted centroid: $\bar{x} = \sum F_i x_i / \sum F_i$
`FLUX_AUTO`	Sum of all fluxes: $F_{\text{total}} = \sum F_i$
`MAG_AUTO`	Recomputed: $m = -2.5 \log_{10}(F_{\text{total}}) + m_0$
`NPIX_ISO`	Sum of all pixel counts
`ISO_RADIUS`	$r = \sqrt{N_{\text{pix,total}}/(\pi \cdot b/a)}$
`A_IMAGE`, `B_IMAGE`, `THETA_IMAGE`	Flux-weighted average
`NUMBER`	New number: $\max(\text{all NUMBERs}) + 1$
Other columns	Copied from the brightest source

After merging:

1. The original sources are removed from `catpanel(alldata)`
2. The merged source row is appended
3. The table and markers are regenerated
4. The merged source is auto-selected and the image pans to its position
5. Status bar: “Merged N sources into #NEW: pos=(X,Y) mag=M”

8.5.3 Cancel (Escape)

Pressing **Escape** cancels the merge selection, removes all red markers, and clears the merge list.

8.6 Trim — Column Filter Dialog

The **Trim...** button opens a modal dialog for per-column min/max filtering.

8.6.1 Dialog Layout

Trim dialog structure

```
+-------------------------------------------+
|  Trim / Column Filter                     |
+-------------------------------------------+
|  Column         |  Min      |  Max        |
|  NUMBER         | [______]  | [______]    |
|  X_IMAGE        | [______]  | [______]    |
|  Y_IMAGE        | [______]  | [______]    |
|  ...            | ...       | ...         |
|  MAG_AUTO       | [______]  | [______]    |
```

```
+-------------------------------------------+
|   [Apply] [Reset] [Save] [Load] [Cancel]   |
+-------------------------------------------+
```

- Empty fields mean no constraint
- **Apply**: filter rows where min $\leq$ value $\leq$ max for each specified column, then close dialog and regenerate markers
- **Reset**: clear all min/max fields
- **Save/Load**: persist conditions to `~/.ds9/sextract_trim.prf`
- **Cancel**: close without changes

Status bar after Apply: "Trimmed: N of M sources match conditions."

8.7 Settings Dialog

The **Settings...** button opens a modal dialog to adjust extraction parameters before running Extract.

Parameter	**Default**	**Description**
Detect Threshold	1.5	Detection threshold (σ)
Detect Min Area	5	Minimum connected pixels
Deblend NThresh	32	Number of deblending sub-thresholds
Deblend MinCont	0.005	Minimum deblending contrast
Phot Aperture	5.0	Fixed aperture diameter (pixels)
Phot Aperture 2	4.0	2nd circular aperture diameter (pixels)
Phot Aperture 3	6.0	3rd circular aperture diameter (pixels)
Phot Aperture 5	10.0	4th circular aperture diameter (pixels)
Mag Zeropoint	25.0	Magnitude zero-point m_0
Gain	0.0	Detector gain (e^-/ADU)
Pixel Scale	1.0	Pixel scale (arcsec/pixel)
Seeing FWHM	3.0	Seeing FWHM (arcsec)
Back Size	64	Background mesh size (pixels)
Back Filter Size	3	Background filter kernel (mesh cells)
Conv Filter	default	Detection convolution filter (default, gauss5x5, mexhat, tophat)

Settings are saved to / loaded from `~/.ds9/sextract.prf`. The **Defaults** button restores all parameters to their default values.

8.8 Add Objects (Click + A)

The Add Objects feature enables manual detection and addition of individual sources that were missed by the automatic extraction. This is useful for faint sources near the detection threshold or sources in complex environments (e.g., near bright stars, in galaxy halos, or at the edge of the field).

8.8.1 Activation

Enable via **Display** → **Add Objects (Click + A)**. This toggles a checkbutton in the Display menu; when enabled, the `a` key triggers source addition.

8.8.2 Workflow

1. Enable "Add Objects" in the Display menu.
2. Click on an empty area of the image where a source should be detected.
3. Press `a`. The system:
   (a) Saves the FITS image to a temporary file.
   (b) Runs `ds9_add_source.py` with the saved image coordinates.
   (c) Extracts a 25×25 pixel cutout centered on the click position.
   (d) Runs SEP detection with aggressive parameters (threshold 0.8σ, min area 3 px).
   (e) Finds the nearest detected source within 10 px of the click position.
   (f) Appends the source to the catalog and regenerates markers.
4. The status bar shows the new source's NUMBER and position.

8.8.3 Detection Parameters

Add Objects uses more aggressive detection parameters than the standard extraction to maximize sensitivity near the click position:

Parameter	**Standard**	**Add Objects**
Detect threshold	1.5σ	0.8σ
Minimum area	5 px	3 px
Deblend nthresh	32	64
Deblend mincont	0.005	0.0001
Cutout radius	full image	25 px
Max distance	—	10 px

> The click position is recorded in image coordinates (not canvas coordinates) at the moment of the click. This ensures correct positioning even after panning or zooming the image between the click and the key press.

Script: `ds9_add_source.py`

8.9 Delete Objects (Click + D)

The Delete Objects feature removes a selected source from both the catalog table and the image markers.

8.9.1 Workflow

1. Click on a source's ellipse marker to select it (or click a row in the catalog table).
2. Press `d`. The system:
   (a) Identifies the selected source by its NUMBER.
   (b) Removes the corresponding row from the internal catalog data (`catpanel(alldata)`).

   (c) Deletes the source's marker from the image.
   (d) Reloads the catalog table and regenerates all markers.
3. The status bar shows "Deleted source N".

Also available via **Display** → **Delete Selected (Click + D)**.

Important

Deletion is irreversible within the current session. Save the catalog before deleting sources if you need to preserve the original data.

8.10 Source Separation (Click + S)

The Separate feature deblends a selected source into multiple sub-components using SEP with aggressive deblending parameters. This is essential for resolving blended sources in crowded fields or merging galaxies.

8.10.1 Workflow

1. Click on a source's ellipse marker to select it.
2. Press `s`. The system:
   (a) Reads the selected source's position, shape (A_IMAGE, B_IMAGE, THETA_IMAGE), and ISO_RADIUS.
   (b) Saves the FITS image temporarily.
   (c) Runs `ds9_separate.py` with the source parameters.
   (d) Extracts a cutout of radius $3 \times \max(a,b)$ around the source.
   (e) Runs SEP with aggressive deblending (nthresh=64, mincont=0.0001).
   (f) Filters sub-sources to keep only those within the parent's ellipse region.
   (g) If ≥ 2 sub-components are found, replaces the parent row with sub-source rows.
3. Sub-sources are numbered as `parent.1`, `parent.2`, etc.

8.10.2 Brightest Sub-Source Shape Preservation

The brightest sub-source (by flux) retains the parent's original ellipse parameters:

- `A_IMAGE`, `B_IMAGE`, `THETA_IMAGE` — copied from parent
- `ISO_RADIUS` — copied from parent (ensures correct marker rendering)

This preserves the morphological characterization of the dominant component while splitting off fainter companions with independently measured shapes.

8.10.3 Separate Settings

Accessible via **Separate** → **Settings**. Parameters are saved to / loaded from `~/.ds9/sextract.prf`:

Parameter	**Default**	**Description**
Deblend NThresh	64	Number of deblending sub-thresholds
Deblend MinCont	0.0001	Minimum contrast for deblending
Detect Threshold	0.8	Detection threshold (σ)
Detect Min Area	3	Minimum connected pixels
Radius Factor	3.0	Cutout radius = factor $\times \max(a,b)$
Back Size	32	Background mesh size (pixels)

Script: `ds9_separate.py`

8.11 PSF Star Management

PSF stars are marked with **purple dashed circle markers** in the image. They can be managed through the Star(PSF) menu.

8.11.1 Star Finding Methods

Method	Description
Combined	Multi-criteria selection using CLASS_STAR, FWHM, ellipticity, and signal-to-noise ratio (recommended)
CLASS_STAR	Select by star/galaxy classifier output (CLASS_STAR > threshold)
FWHM	Select by FWHM consistency with seeing

8.11.2 PSF Construction Methods

Method	Description
Median	Pixel-wise median stack of star cutouts (robust to outliers)
Mean	Pixel-wise mean stack (higher S/N)
ePSF	Effective PSF via iterative sub-pixel centering and stacking
Gaussian	Analytical 2D Gaussian fit to the stacked profile
Moffat	Analytical 2D Moffat fit (better for wide wings)

Stars can be manually removed via **Ctrl+Click** on a purple star marker. The PSF is saved as a FITS file (`~/.ds9/psf_current.fits`).

8.12 Marker Tag Reference

Tag	Description
`sextract_all`	All yellow ellipse markers from Mark All
`sextract_src.N`	Individual source marker (N = source NUMBER)
`sextract_sel`	Current selection markers (cyan cross + green ellipse)
`sextract_merge`	All red merge-candidate markers
`sextract_merge.N`	Individual merge-candidate marker
`ai_merge`	AI Merge group visualization markers
`psf_star`	Purple circle markers for identified PSF stars

8.13 Keyboard Shortcuts and Mouse Interactions

All interactive features use a "click + key" paradigm: click on the image to select a position or source, then press a key to execute the action.

8.13.1 Mouse Actions

Action	Effect
Click on marker	Navigate catalog to the corresponding source, show cyan cross + green ellipse selection markers, pan image to source position
Click on empty area	Save image coordinates at click position (for Add/Delete/Separate operations)
Ctrl+Click on marker	Toggle merge candidate selection (red thick ellipse overlay). Click again to deselect.
Ctrl+Click on PSF star	Remove the PSF star marker (purple circle) and deselect the star from PSF building
Click table header	Sort catalog by that column (toggle ascending/descending)
Click table row	Pan image to the source position, show selection markers

8.13.2 Single-Key Shortcuts (after click)

These shortcuts operate on the position or source selected by the most recent mouse click. No modifier key is required.

Key	Action	Description
`a`	Add Object	Detect and add a new source at the last-clicked position. Requires "Add Objects" to be enabled in the Display menu. Uses SEP to find the nearest source within 10 px of the click position and appends it to the catalog.
`s`	Separate Source	Deblend the currently selected source into sub-components using aggressive SEP deblending. The brightest sub-source retains the parent's original ellipse shape. Sub-sources are numbered as `parent.1`, `parent.2`, etc.
`d`	Delete Source	Remove the currently selected source from both the catalog table and the image markers. Irreversible within the current session (use Save before deleting).

The click + key paradigm was chosen over Ctrl+Key combinations because:
- Ctrl+S and Ctrl+A are reserved by the system (File Save, Select All)
- The two-step interaction (click, then key) provides a confirmation step that reduces accidental operations
- It is ergonomically more comfortable for repeated use

8.13.3 Modifier-Key Shortcuts

Key	Action	Description
Ctrl+M	Merge Sources	Merge all selected merge candidates (red ellipses) into a single source. Requires ≥ 2 candidates selected via Ctrl+Click. Flux-weighted centroid, summed flux, recomputed magnitude.
Escape	Cancel / Done	Cancel merge selection (clear red markers), or exit AI Merge navigation mode.

8.13.4 AI Merge Navigation Keys

These keys are active only during AI Merge review mode (after running AI Merge from the menu).

Key	Action	Description
`n` or Right	Next Group	Navigate to the next merge candidate group
`p` or Left	Previous Group	Navigate to the previous merge candidate group
`a`	Accept Group	Accept the current merge candidate (merge the group)
`r`	Reject Group	Reject the current merge candidate (keep sources separate)
Escape	Done	Exit AI Merge review mode

8.13.5 Quick Reference Card

DS9 Source Extractor — Quick Reference	
	Source Editing
Click + `a`	Add source at click position
Click + `s`	Separate (deblend) selected source
Click + `d`	Delete selected source
	Source Merging
Ctrl+Click	Toggle merge candidate
Ctrl+M	Execute merge
Escape	Cancel merge
	PSF Stars
Ctrl+Click on star	Remove PSF star
	AI Merge Review
n / Right	Next group
p / Left	Previous group
a	Accept merge
r	Reject merge
Escape	Exit review

9 Advanced Analysis Features

The **Analysis** menu provides advanced photometric, morphological, and catalog analysis tools implemented as Python scripts. Each tool reads the current catalog and FITS image, performs its analysis, and appends result columns to the catalog table.

9.1 Non-Parametric Morphology (CAS/Gini/M20)

Measures five non-parametric morphological indices for each source, following the definitions of Conselice (2003), Lotz et al. (2004), and Petrosian (1976).

Index	Definition
CONC	Concentration index: $C = 5\log_{10}(r_{80}/r_{20})$, where r_{80} and r_{20} are the circular radii enclosing 80% and 20% of the total flux (computed via `sep.flux_radius`).
ASYM	Asymmetry: $A = \min\big(\sum\lvert I - I_{180}\rvert\big)/(2\sum\lvert I\rvert)$, where I_{180} is the image rotated 180° about the center. The center is optimized to minimize A.
GINI	Gini coefficient of the pixel intensity distribution within the segmentation footprint: $G = \frac{1}{\bar{x}n(n-1)}\sum_{i=1}^{n}(2i-n-1)x_i$, where x_i are the sorted pixel values.
M20	$M_{20} = \log_{10}\big(\sum_i f_i r_i^2 \text{ (brightest 20\%)}/\sum_i f_i r_i^2\big)$, a second-order moment of the brightest 20% of pixels.
R_PETRO	Petrosian radius: the radius r where $\eta(r) = I(r)/\langle I(<r)\rangle = 0.2$, computed from concentric circular annuli.

Package: `morphometry/` — modules: `concentration.py`, `asymmetry.py`, `gini.py`, `m20.py`, `petrosian.py`, `measure.py`.

CLI: `ds9_morphometry.py FITSFILE -catalog catalog.tsv [-min-npix 100]`

Dependencies: `sep`, `scipy.ndimage`, `astropy.io.fits`

9.2 Sérsic Profile Fitting

Fits a 2D Sérsic profile to each source:

$$I(r) = I_e \exp\left[-b_n\left(\left(r/r_e\right)^{1/n} - 1\right)\right] \tag{9.1}$$

where n is the Sérsic index, r_e is the effective (half-light) radius, I_e is the intensity at r_e, and $b_n \approx 2n - 1/3 + 4/(405n)$.

Column	Description
`SERSIC_N`	Sérsic index n (constrained $0.2 < n < 10$). $n = 1$: exponential disk, $n = 4$: de Vaucouleurs profile.
`SERSIC_RE`	Effective radius r_e in pixels.
`SERSIC_IE`	Surface brightness at r_e.
`SERSIC_ELLIP`	Fitted ellipticity $e = 1 - b/a$.
`SERSIC_THETA`	Position angle of the fitted ellipse (degrees).
`SERSIC_CHI2`	Reduced χ^2 of the fit.

The fitting uses `scipy.optimize.least_squares` (Levenberg–Marquardt) with 8 free parameters: I_e, r_e, n, x_c, y_c, ellipticity, θ, and background level.

Package: `sersic_fit/` — modules: `models.py`, `fitter.py`, `utils.py`.

CLI: `ds9_sersic.py FITSFILE -catalog catalog.tsv [-max-sources 200]`

9.3 PSF Photometry

Performs point-source PSF photometry by fitting a scaled, shifted PSF model to each source.

For each source, the model is:

$$M(x,y) = A \cdot \mathrm{PSF}(x - x_0 - \Delta x,\ y - y_0 - \Delta y) + B \tag{9.2}$$

The four free parameters (A, Δx, Δy, B) are optimized via `scipy.optimize.least_squares`. The fitted flux is $F_{\mathrm{PSF}} = A \cdot \sum \mathrm{PSF}$.

Column	Description
`FLUX_PSF`	PSF-fitted flux
`FLUXERR_PSF`	1σ flux error from covariance
`MAG_PSF`	PSF magnitude
`MAGERR_PSF`	PSF magnitude error
`CHI2_PSF`	Reduced χ^2 of the PSF fit
`X_PSF`, `Y_PSF`	Refined centroid from the fit

Prerequisite: A PSF image must exist (generated via Reconstruction → Build PSF, stored as `~/.ds9/psf_current.fits`).

Package: `psf_phot/` — modules: `fitter.py`, `photometry.py`, `subtract.py`.

CLI: `ds9_psf_phot.py FITSFILE -catalog cat.tsv -psf psf.fits`

9.4 Multi-Band Photometry

Performs matched-aperture photometry across multiple filter images using source positions detected in a single reference (detection) image.

1. Source positions and shapes are determined from the detection image (or existing catalog).
2. For each band image, Kron elliptical and circular aperture photometry is performed at the same positions.
3. Per-band columns are added: `FLUX_AUTO_`*`BAND`*, `MAG_AUTO_`*`BAND`*, `FLUX_APER_`*`BAND`*, etc.
4. Color indices are computed automatically: `COLOR_`*`B1`*`_`*`B2`* $= m_{B1} - m_{B2}$.

A dialog allows adding bands by name and FITS file path.

CLI: `ds9_multiband.py -detect-image deep.fits -bands F435W:f435w.fits,F606W:f606w.fits`

9.5 Crowded Field Photometry (DAOPHOT-Style)

Implements iterative multi-PSF fitting for crowded stellar fields, following the DAOPHOT approach (Stetson 1987):

1. **Grouping**: Sources with overlapping Kron radii are assigned to the same group using adjacency-based graph clustering.
2. **Simultaneous fitting**: For each group, all N source PSFs are fitted simultaneously by minimizing:

$$\chi^2 = \sum_{\text{pixels}} \left(I_{\text{data}} - \sum_{j=1}^{N} A_j \cdot \text{PSF}(x - x_j, y - y_j) - B \right)^2 \tag{9.3}$$

with $3N + 1$ free parameters (A_j, Δx_j, Δy_j per source, plus background B).
3. **Subtraction**: Fitted PSFs are subtracted from the image.
4. **Re-detection**: New sources are detected in the residual image.
5. Steps 1–4 are repeated for a configurable number of iterations (default: 3).

Column	Description
`FLUX_CROWD`	Multi-PSF fitted flux
`FLUXERR_CROWD`	1σ flux error
`MAG_CROWD`	Crowded-field magnitude
`X_CROWD`, `Y_CROWD`	Refined positions
`N_NEIGHBORS`	Number of sources in the fitting group

Prerequisite: A PSF image must exist.

Package: `crowded_phot/` — modules: `group.py`, `nstar.py`, `psf_subtract.py`, `iterate.py`.

CLI: `ds9_crowded_phot.py FITSFILE -catalog cat.tsv -psf psf.fits [-max-iter 3]`

9.6 Catalog Cross-Match (VizieR)

Cross-matches the current catalog against external reference catalogs hosted on VizieR using TAP/ADQL queries.

9.6.1 Supported Catalogs

Name	VizieR ID	Contents
Gaia DR3	I/355/gaiadr3	Astrometry + photometry
2MASS	II/246/out	Near-infrared JHK_s
Pan-STARRS DR1	II/349/ps1	Optical *grizy*
SDSS DR17	V/154/sdss16	Optical *ugriz*
AllWISE	II/328/allwise	Mid-infrared $W1$–$W4$

9.6.2 Algorithm

1. Extract `ALPHA_J2000` and `DELTA_J2000` from the current catalog.
2. Compute the field center and search radius.
3. Query VizieR via TAP: `SELECT * FROM "catalog" WHERE 1=CONTAINS(POINT('ICRS',RA,DEC), CIRCLE('ICRS',`α_c`,`δ_c`,`r`))`.
4. Match each source to the nearest catalog entry within the specified radius (default: 2 arcsec).
5. Append columns: `MATCH_ID`, `MATCH_DIST`.

A dialog allows selecting the catalog and matching radius.

CLI: `ds9_crossmatch.py -catalog cat.tsv -vizier-cat II/349 -radius 2.0`

Requirement: `ALPHA_J2000` and `DELTA_J2000` must be present (requires WCS in the FITS header).

9.7 Dual-Image Mode

Performs source detection on one image and photometric measurement on another, following the SExtractor dual-image paradigm. This is essential for multi-band surveys where consistent source definitions are needed across filters.

1. Source detection (`sep.extract`) is run on the **detection image** (typically a deep or stacked image).
2. Kron radius, aperture photometry, and shape measurements are performed on the **measurement image** at the detected positions.
3. The output catalog uses detection-image positions with measurement-image fluxes and magnitudes.

A dialog allows selecting the detection and measurement images from open DS9 frames or FITS files.

CLI: `ds9_dual_extract.py -detect-image deep.fits -measure-image filter.fits`

9.8 Segmentation Map

Generates a segmentation map where each pixel is labeled with the ID of the source it belongs to (0 = background). The result is loaded as a new DS9 frame with a random colormap for visualization.

CLI: `ds9_segmap.py FITSFILE [-detect-thresh 1.5] [-output ~/.ds9/segmap.fits]`

9.9 Completeness Simulation

Estimates detection completeness as a function of magnitude by injecting artificial sources and attempting to recover them:

1. Inject N artificial PSF sources at random positions (avoiding existing sources), distributed uniformly in magnitude bins from m_{min} to m_{max}.
2. Re-run source extraction with the same parameters.
3. Cross-match injected and recovered sources (< 2 pixel separation).
4. Compute completeness per magnitude bin: $C(m) = N_{\text{recovered}}/N_{\text{injected}}$.

Output: `MAG_BIN \t N_INJECTED \t N_RECOVERED \t COMPLETENESS`

A dialog allows configuring the number of injected sources, magnitude range, and number of bins.

CLI: `ds9_completeness.py FITSFILE [-n-inject 1000] [-mag-min 20] [-mag-max 28]`

9.10 Export Features

9.10.1 Export Regions (.reg)

Exports the current catalog as a DS9 region file. Each source is represented as an ellipse with parameters from `X_IMAGE`, `Y_IMAGE`, `A_IMAGE`, `B_IMAGE`, and `THETA_IMAGE`. Source numbers are included as text labels.

Menu: SExtractor → Export Regions (.reg)

9.10.2 Export FITS Table

Exports the current catalog as a FITS binary table using `astropy.io.fits.BinTableHDU`. Column types are automatically detected (integer, float, or string).

CLI: `ds9_fits_export.py -input catalog.tsv -output catalog.fits`

Menu: SExtractor → Export FITS Table

9.11 Python Dependencies

All Analysis features require Python 3 with the following packages:

Package	Required by	Install
`numpy`	All features	`pip install numpy`
`scipy`	Sérsic, PSF phot, crowded phot, morphometry	`pip install scipy`
`sep`	Morphometry, segmentation, completeness, dual-image	`pip install sep`
`astropy`	FITS export, cross-match, all FITS I/O	`pip install astropy`

10 Building ds9_sextract for Each Platform

The `ds9_sextract` binary is a standalone C program that uses the SEP library and CFITSIO for source extraction. It must be compiled separately for each target platform and placed in the `bin/` directory alongside the `ds9` executable.

10.1 Dependencies

Library	Purpose	Install
CFITSIO	FITS file I/O	Package manager or source
SEP	Source extraction algorithm	Included in `ds9_sextract.c` or separate
libm	Math functions	System standard

10.2 Linux (x86_64)

Install dependencies (Ubuntu/Debian)

```
sudo apt-get install libcfitsio-dev
```

Install dependencies (RHEL/CentOS/Rocky)

```
sudo dnf install cfitsio-devel
```

Install dependencies (conda)

```
conda install -c conda-forge cfitsio
```

Compile

```
gcc -O2 -o ds9_sextract ds9_sextract.c \
    -I/usr/include -L/usr/lib64 \
    -lcfitsio -lm
# Or with conda:
gcc -O2 -o ds9_sextract ds9_sextract.c \
    -I$CONDA_PREFIX/include -L$CONDA_PREFIX/lib \
    -lcfitsio -lm -Wl,-rpath,$CONDA_PREFIX/lib
cp ds9_sextract /path/to/SAOImageDS9/bin/
```

10.3 macOS (Intel and Apple Silicon)

Install dependencies (Homebrew)

```
brew install cfitsio
```

Install dependencies (conda)

```
conda install -c conda-forge cfitsio
```

Compile (Homebrew)

```
# Intel Mac:
gcc -O2 -o ds9_sextract ds9_sextract.c \
    -I/usr/local/include -L/usr/local/lib \
    -lcfitsio -lm

# Apple Silicon (M1/M2/M3):
gcc -O2 -o ds9_sextract ds9_sextract.c \
    -I/opt/homebrew/include -L/opt/homebrew/lib \
    -lcfitsio -lm
```

Compile (conda)

```
gcc -O2 -o ds9_sextract ds9_sextract.c \
    -I$CONDA_PREFIX/include -L$CONDA_PREFIX/lib \
    -lcfitsio -lm -Wl,-rpath,$CONDA_PREFIX/lib
```

Place binary

```
cp ds9_sextract /path/to/SAOImageDS9/bin/
```

Important

On macOS, if you encounter "library not loaded" errors at runtime, ensure the CFITSIO library path is embedded using `-Wl,-rpath` or set `DYLD_LIBRARY_PATH` before launching DS9.

10.4 Windows

10.4.1 Using MSYS2/MinGW-w64

Install MSYS2 and dependencies

```
# In MSYS2 MinGW64 shell:
pacman -S mingw-w64-x86_64-gcc mingw-w64-x86_64-cfitsio
```

Compile

```
gcc -O2 -o ds9_sextract.exe ds9_sextract.c \
    -I/mingw64/include -L/mingw64/lib \
    -lcfitsio -lm
```

Deploy

```
# Copy binary and required DLLs to ds9 bin directory
cp ds9_sextract.exe /path/to/SAOImageDS9/bin/
cp /mingw64/bin/libcfitsio*.dll /path/to/SAOImageDS9/bin/
cp /mingw64/bin/zlib1.dll /path/to/SAOImageDS9/bin/
```

10.4.2 Using Visual Studio

Compile with MSVC

```
cl /O2 /Fe:ds9_sextract.exe ds9_sextract.c ^
   /I"C:\cfitsio\include" ^
   /link /LIBPATH:"C:\cfitsio\lib" cfitsio.lib
```

Important

On Windows, required DLLs (`libcfitsio*.dll`, `zlib1.dll`) must be placed in the same directory as `ds9_sextract.exe`, or their location must be in the system `PATH`.

10.5 Cross-Platform Tcl Integration

The `CatalogPanelExtract` procedure in `layout.tcl` automatically handles platform differences:

Platform-aware binary discovery

```
set os $::tcl_platform(os)
if {$os eq "Windows NT"} {
    set sextract [file join $bindir ds9_sextract.exe]
} else {
    set sextract [file join $bindir ds9_sextract]
}
```

Platform-aware library path

```
if {$os eq "Darwin"} {
    # macOS: set DYLD_LIBRARY_PATH
    set ::env(DYLD_LIBRARY_PATH) [join $libpaths :]
} elseif {$os ne "Windows NT"} {
    # Linux/Unix: set LD_LIBRARY_PATH
    set ::env(LD_LIBRARY_PATH) [join $libpaths :]
}
# Windows: DLLs found via PATH automatically
```

Cross-platform execution

```
# Direct exec without sh -c (works on all platforms)
exec $sextract $fn {*}$paramargs 2>@stderr
```

	Linux	macOS	Windows
Binary name	`ds9_sextract`	`ds9_sextract`	`ds9_sextract.exe`
Library path env	`LD_LIBRARY_PATH`	`DYLD_LIBRARY_PATH`	`PATH`
Shell required	No	No	No

10.6 Verification

After building for any platform, verify the binary works:

Test extraction

```
./ds9_sextract test_image.fits | head -5
```

Expected output: a tab-separated header line followed by data rows. If library errors occur, check the library path configuration for your platform.

11 Planned Features and Science-Driven Extensions

This chapter outlines features motivated by upcoming survey science requirements, particularly the Vera C. Rubin Observatory Legacy Survey of Space and Time (LSST) and low surface brightness (LSB) science such as Intra-Cluster Light (ICL) detection.

11.1 LSST / Large-Scale Survey Features

The LSST will produce ∼20 TB of imaging data per night across six filters (*ugrizy*), resulting in ∼60 PB over its 10-year survey. Processing and inspecting such data volumes requires features beyond single-image analysis.

11.1.1 Batch Processing Pipeline

Process multiple FITS files in sequence or parallel from the command line, with per-file output catalogs:

Batch mode

```
# Process all FITS files with 8 parallel workers
python3 ds9_morphometry.py --batch field*.fits \
    --catalog cat.tsv --n-workers 8 --output-dir results/

# Future MPI extension (no code changes needed):
mpirun -np 16 python3 ds9_morphometry.py --batch *.fits \
    --output-dir results/
```

Status: Designed (see `parallel/` package plan). Adds `-n-workers` and `-batch` arguments to all CLI scripts.

11.1.2 Multi-Extension FITS (MEF) Support

LSST CCDs produce Multi-Extension FITS files with one extension per amplifier or detector. Features needed:

- Auto-detect and iterate over all image extensions
- Per-extension background subtraction and source extraction
- Unified catalog with extension ID column
- Mosaic assembly for visualization

11.1.3 Tile/Tract-Based Catalog Management

LSST data products are organized as sky tiles (tracts/patches). Features needed:
- Load catalogs from Butler data repositories or Parquet files
- Spatial indexing (HEALPix, HTM) for efficient viewport queries
- Cross-tract deduplication within overlap regions
- Catalog streaming — load only sources within the current viewport for catalogs with $> 10^6$ sources

11.1.4 Difference Imaging Transient Detection

LSST alert production relies on image differencing. Features needed:
- **Template subtraction**: Subtract a template (coadd) image from a visit image using kernel matching (e.g., Alard & Lupton 1998, ZOGY)
- **Dipole detection**: Detect positive/negative dipole pairs from astrometric misalignment or moving objects
- **Transient classification**: Real/bogus classification of difference-image detections using a pre-trained CNN
- **Light curve viewer**: Plot photometric history of a selected transient across epochs

11.1.5 Forced Photometry

Measure flux at fixed positions across multiple epochs or bands, even when the source is below the detection threshold:
- Input: reference catalog positions + per-epoch FITS images
- Output: per-epoch flux measurements with uncertainties
- Essential for faint variable sources and upper limit estimation

11.1.6 Quality Assurance Dashboard

For survey-scale data, visualize data quality across fields:
- Seeing FWHM, sky background, and zero-point maps across the focal plane
- Source count density and completeness maps
- Photometric residual plots (catalog − reference)
- Astrometric residual quiver plots

11.2 Intra-Cluster Light (ICL) Detection

ICL is the diffuse stellar component in galaxy clusters, typically at surface brightness levels of $\mu_V \gtrsim 26\,\mathrm{mag/arcsec^2}$. Detecting and characterizing ICL requires specialized techniques beyond standard source extraction.

11.2.1 Low Surface Brightness (LSB) Mode

A dedicated extraction mode optimized for extended, faint emission:
- **Large-scale background model**: Use back_size $\gg 256\,\mathrm{px}$ or 2D polynomial/spline fit to avoid subtracting the ICL signal as "background"
- **Multi-scale detection**: Wavelet-based detection (e.g., à trous algorithm) to separate ICL from compact sources and background fluctuations

- **Aggressive masking**: Mask all detected compact sources (stars, galaxies) and their extended halos before measuring the diffuse component
- **Surface brightness profiles**: Radial SB profile centered on the BCG (brightest cluster galaxy) with azimuthal averaging and sector-based analysis

11.2.2 ICL-Specific Measurements

Measurement	Description
ICL fraction	$f_{\rm ICL} = L_{\rm ICL}/(L_{\rm ICL} + L_{\rm galaxies})$ within a defined radius (e.g., R_{500})
SB profile	Azimuthally averaged surface brightness profile $\mu(r)$ from the BCG center, extending to $r \sim 500$ kpc
Color gradient	ICL color profile across multiple bands, constraining the stellar population age and metallicity
Isophotal radius	Radius at which the SB profile reaches specified isophotes (e.g., $\mu_V = 26, 27, 28$ mag/arcsec^2)
Tidal features	Detection and characterization of tidal streams, shells, and plumes associated with the ICL

11.2.3 Source Masking for Diffuse Light

Before measuring ICL, compact sources must be aggressively masked:

1. Run standard source extraction to build the segmentation map.
2. Expand each segment by a factor of 2–3$\times$ in radius to mask extended wings.
3. Bright stars: mask with magnitude-dependent circular masks (from a star catalog or saturation-based).
4. Fill masked regions via 2D interpolation (or median of surrounding pixels) for smooth background estimation.
5. Optionally iterate: re-detect LSB features in the masked image, update the mask, and repeat.

11.2.4 Background Modeling Strategies for ICL

Standard SExtractor-style mesh-based backgrounds will subtract ICL signal. Alternative approaches:

Method	Description
Global polynomial	Fit a low-order 2D polynomial to masked-source image; residual contains ICL
Iterative clipping	Iteratively estimate background using σ-clipped median in large (~ 500 px) cells, excluding source regions
Ring median	For each radial bin from the cluster center, compute the median of unmasked pixels as the local background

Method	Description
DBSCAN clustering	Use unsupervised clustering on pixel coordinates and intensities to separate ICL from sky background

11.3 Additional Feature Ideas

11.3.1 Photometric Redshift Estimation

Estimate photometric redshifts from multi-band photometry using template fitting or machine learning:

- Template fitting: fit SED templates (e.g., from LePhare or EAZY) to multi-band magnitudes
- ML approach: train a neural network on spectroscopic redshift calibration samples
- Output columns: `ZPHOT_BEST`, `ZPHOT_ERR`, `ZPHOT_PDF` (probability distribution)
- Critical for LSST cosmology: weak lensing, BAO, cluster mass calibration

11.3.2 Weak Lensing Shape Measurement

Measure galaxy shapes for weak gravitational lensing analysis:

- PSF deconvolution of galaxy shapes using the built-in PSF model
- Ellipticity components (e_1, e_2) and shear responsivity
- Model fitting: Gaussian or Sérsic convolved with PSF
- PSF interpolation across the field of view
- Essential for LSST dark energy science

11.3.3 Spectral Energy Distribution (SED) Viewer

Interactive SED plotting from multi-band photometry:

- Click a source to display its SED (flux vs. wavelength) in a popup window
- Overlay best-fit template from photometric redshift fitting
- Color-coded data points per filter
- Integrated with Multi-Band Photometry results

11.3.4 Catalog Annotation and Flagging

Manual annotation tools for visual inspection workflows:

- Add custom text notes to individual sources
- Flag sources as "artifact", "interesting", "needs review"
- Export flagged source lists for follow-up observation planning
- Useful for citizen science and expert review of LSST alerts

11.3.5 Cluster Finder

Automated galaxy cluster detection algorithms:

- Red-sequence cluster finder (RedMaPPer-style) using multi-band colors
- Friends-of-friends clustering in projected space
- Voronoi tessellation density mapping
- Richness estimation and membership probability

11.3.6 Parallel Processing

Multi-core parallelization using `multiprocessing.shared_memory` for CPU-intensive analyses:

- Source-level parallelism for morphometry, Sérsic fitting, PSF photometry
- Group-level parallelism for crowded field photometry
- Band-level parallelism for multi-band photometry
- Shared memory arrays avoid data copying overhead
- `-n-workers` CLI argument: 0=auto (cpu−1), 1=sequential, N=fixed

Status: Designed (see `parallel/` package plan).